

SE801 Project Midterm Technical Report

Submitted by: [Your Name] ([Your Student ID]) **Supervised by:** [Supervisor's Name] **Submission Date:** [Submission Date]

Letter of Transmittal

[Submission Date]

[Exam / Project Committee]

[Department / Institute Name]

Subject: Technical Report Submission — VisionGuard AI: A CPU-Friendly CCTV Surveillance Platform with ROI-Based Hybrid Dehazing

Sir/Madam,

I am submitting the technical report for VisionGuard AI, prepared as part of the SE801 project. This report documents the project's requirements, system models, data design, and preliminary test plan as developed to date. I have made every effort to present the work accurately and welcome any feedback for improvement.

Thank you for your kind consideration.

Sincerely,

[Your Name]

[Your Student ID]

[Submission Date]

----- Supervisor's Signature

Acknowledgement

I would like to thank my supervisor, [Supervisor's Name], for the guidance and feedback provided throughout the development of VisionGuard AI. Their input on scoping the hybrid dehazing architecture and on keeping the system CPU-friendly and explainable has directly shaped the design decisions documented in this report.

Abstract

VisionGuard AI is a lightweight, CPU-friendly CCTV surveillance platform that restores visibility in hazy or smoggy footage in real time. Rather than relying on an end-to-end deep learning model, it uses a hybrid approach: the Dark Channel Prior (DCP) produces a physics-based estimate of a scene's transmission map, and a small refinement CNN corrects that estimate before the clean image is reconstructed through the atmospheric scattering model. The system's central optimization is region-of-interest (ROI) processing — dehazing runs only on the portion of a frame flagged by motion detection, rather than on every pixel of every frame, which keeps the pipeline fast enough for real-time use on ordinary CPU hardware and suitable for edge deployment without a GPU.

Beyond the dehazing pipeline itself, VisionGuard AI supports live webcam and IP camera feeds as well as uploaded video files, automatically logs every enhanced event with its timestamp, snapshot, and region coordinates, and lets operators review that history through both a chronological timeline and a natural-language search interface backed by the Claude API. Access is restricted to admin-seeded operator accounts authenticated with JWTs, with no open self-registration. The target users are security guards and surveillance operators in settings such as campuses, warehouses, and parking lots, where hazy conditions can otherwise leave footage unusable exactly when it matters most.

Table of Contents

- Letter of Transmittal
- Acknowledgement
- Abstract
- 1. Project Overview
 - 1.1 Project Title
 - 1.2 Problem Statement
 - 1.3 Objectives
 - 1.4 Scope
 - 1.5 Deliverables
- 2. Requirements Analysis
 - 2.1 Functional Requirements
 - 2.2 Non-functional Requirements
 - 2.3 Stakeholders
- 3. System Modeling
 - 3.1 Use Case Diagram
 - 3.1.1 VisionGuard AI (Level 0)
 - 3.1.2 Modules of VisionGuard AI (Level 1)
 - 3.1.3 User Management
 - 3.1.4 Camera & Feed Management
 - 3.1.5 Motion Detection & ROI Extraction
 - 3.1.6 Hybrid Dehazing Pipeline
 - 3.1.7 Event Logging
 - 3.1.8 Timeline / Event Browser
 - 3.1.9 NL Event Search
 - 3.2 Activity Diagram
 - 3.2.1 Frame Processing Loop (Overview)
 - 3.2.2 User Management

- 3.2.3 Camera & Feed Management
- 3.2.4 Motion Detection & ROI Extraction
- 3.2.5 Hybrid Dehazing Pipeline
- 3.2.6 Event Logging
- 3.2.7 Timeline / Event Browser
- 3.2.8 NL Event Search
- 4. Data & Information Modeling
 - Option A — ER Diagram
 - Option B — Dataset Description
- 5. Preliminary Test Plan
 - 5.1 Testing Objectives
 - 5.2 Features to be Tested

1. Project Overview

1.1 Project Title

VisionGuard AI — A CPU-Friendly CCTV Surveillance Platform with ROI-Based Hybrid Dehazing

1.2 Problem Statement

CCTV footage from surveillance cameras degrades sharply in haze, smog, and fog, which is exactly when security teams need it most — visibility loss during these conditions can hide intrusions, accidents, and other events a surveillance system exists to catch. Modern image dehazing research has moved toward end-to-end deep learning models that produce strong results, but those models are typically too heavy to run on the CPU-only hardware found in most guard rooms and edge installations, and running full-frame enhancement on every frame of a live feed is wasteful when most of a frame is static background with nothing happening in it. VisionGuard AI addresses this by combining a physics-based dehazing model (the Dark Channel Prior) with a lightweight CNN that only refines the prior's transmission estimate, and by restricting all of that computation to the small region of a frame where motion was actually detected — keeping the system real-time, explainable, and deployable without a GPU.

1.3 Objectives

- Recover clear visibility in hazy or smoggy CCTV footage without requiring specialized hardware.
- Keep the dehazing pipeline CPU-friendly and capable of real-time (30 FPS target) operation by only processing regions flagged by motion detection, not entire frames.
- Preserve a hybrid, explainable architecture — physics prior (DCP) plus a lightweight CNN refinement — rather than an opaque end-to-end deep learning model.

- Automatically log every enhanced event (timestamp, snapshot, event type, ROI coordinates) so operators have a retrievable record without manual effort.
- Let operators retrieve past events through both a chronological timeline and a natural-language search interface.
- Restrict system access to authenticated operator and administrator accounts, with no open self-registration.

1.4 Scope

VisionGuard AI's MVP scope covers the full real-time pipeline from camera ingestion through event retrieval:

In scope (MVP):

- Live webcam / IP camera feed ingestion, plus uploaded video file processing.
- Motion detection that skips dehazing entirely when no motion is present.
- ROI extraction from the motion mask, so only the suspicious region is processed.
- The hybrid DCP + Tiny CNN dehazing pipeline (dark channel → Top-K atmospheric light → coarse transmission → CNN refinement → radiance recovery → gamma correction → merge back into frame).
- Event logging with timestamp, snapshot image, event type, and ROI coordinates.
- A timeline / event browser for chronological review.
- Natural language event search (e.g. "any motion last night?") backed by the Claude API.
- JWT-based authentication with admin-seeded accounts (no open signup).

Out of scope (future work, not implemented unless explicitly requested):

- Object detection / intrusion classification.
- ONNX Runtime inference.
- FFmpeg integration.
- Multi-camera support.
- Temporal transmission smoothing across frames.

1.5 Deliverables

- A FastAPI backend implementing the motion-gated, ROI-based hybrid dehazing pipeline.
 - A trained Tiny CNN transmission-refinement model with saved weights (`backend/weights/tiny_cnn.pth`).
 - A PostgreSQL-backed event store with repository-layer access for cameras/video sources and events.
 - A JWT authentication system with admin-seeded accounts and operator account management.
 - A React + TypeScript dashboard for live feed viewing, event timeline browsing, and natural-language event search.
 - A preliminary automated test suite covering the dark channel, transmission estimation, CNN inference, motion detection, and ROI extraction modules.
-

2. Requirements Analysis

2.1 Functional Requirements

- **FR-1:** The system shall allow an operator to start a live feed from a webcam or IP camera, or upload a recorded video file for processing.
- **FR-2:** The system shall allow an operator to stop an active feed at any time and view the latest processed frame while a feed is running.
- **FR-3:** The system shall continuously monitor incoming frames for motion and skip the dehazing pipeline entirely when no motion is detected.
- **FR-4:** When motion is detected, the system shall extract only the region of interest (ROI) containing that motion rather than processing the full frame.
- **FR-5:** The system shall dehaze the extracted ROI using the Dark Channel Prior for an initial transmission estimate, refine that estimate with a Tiny CNN, and reconstruct the clean region via the atmospheric scattering model before merging it back into the original frame.

- **FR-6:** The system shall automatically log every dehazing event with its timestamp, a snapshot image, the event type, and the ROI coordinates.
- **FR-7:** The system shall allow an operator to browse logged events in chronological order on an Events page, showing each event's snapshot and details.
- **FR-8:** The system shall allow an operator to enter a plain-English query (e.g. "any motion last night?") that is parsed into structured filters and used to retrieve matching logged events; if no time range can be determined, the system shall return all events rather than failing.
- **FR-9:** The system shall allow an operator to check current system status to confirm the camera feed and dehazing pipeline are running correctly.
- **FR-10:** The system shall require authentication (JWT) for every endpoint except login; an admin account is seeded on first startup, and only the admin can create operator accounts.

2.2 Non-functional Requirements

- **Performance:** The pipeline must sustain a target of 30 FPS on CPU-only hardware by dehazing only the motion ROI, not the full frame, and by skipping the pipeline entirely on static frames.
- **Security:** Passwords are hashed with bcrypt via passlib and never stored or logged in plaintext; tokens are signed and verified with PyJWT; authentication is enforced only at the router layer via a `get_current_user` dependency, never inside services or controllers.
- **Reliability:** Every module raises meaningful, typed exceptions instead of silently swallowing errors, and model weights are loaded once at startup rather than per-request, so a missing or corrupt weights file fails fast and visibly.
- **Scalability:** Business logic is isolated in stateless service modules with dependency-injected pipeline/camera state (no global mutable state), so the system can be extended to additional camera sources without redesigning the core pipeline.
- **Maintainability:** The codebase follows a strict layered architecture (routers → controllers → services → repositories) with Pydantic schemas for every

request/response and no ORM models leaking into the API layer, keeping responsibilities isolated and the system easy to extend.

- **Usability:** Operators interact with a single dashboard for live viewing, timeline browsing, and natural-language search, with all server state managed through React Query so the UI reflects backend state without manual refresh logic.

2.3 Stakeholders

- **Operators** — security guards, building surveillance operators, university campus surveillance staff, and warehouse/parking-lot monitoring teams who run live feeds, review events, and search the timeline day to day.
 - **Administrator** — manages operator accounts (the only account-creation path in the system) and has full access to all operator-level functionality.
 - **Project Supervisor** — the academic supervisor for SE801, who evaluates the project's technical design, scope adherence, and progress against this report.
-

3. System Modeling

Scenario-based modeling is used here to depict how VisionGuard AI behaves under real usage scenarios. A use case diagram visually represents how the system interacts with its actors to achieve specific goals, showing the relationships between use cases (system functionalities) and the actors (users or systems) that trigger them. Following this convention, the use case diagrams for VisionGuard AI are broken into a Level 0 overview, a Level 1 module breakdown, and Level 2 diagrams for each individual module.

3.1 Use Case Diagram

3.1.1 VisionGuard AI (Level 0)

Level: 0 Use Case ID: 0 Name: VisionGuard AI: Detect, Enhance and Search
Primary Actor: Admin, Operator **Secondary Actor:** None

Level: 0

USE CASE ID: 0

Name: VisionGuard AI: Detect, Enhance and Search

Primary Actor: Admin, Operator

Secondary Actor: None

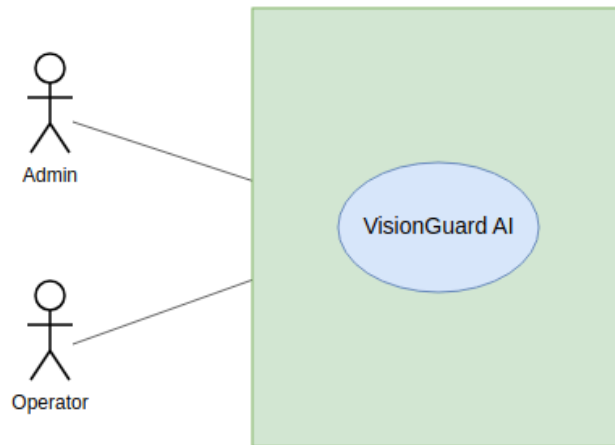


Fig1: VisionGuard AI: Detect, Enhance and Search

Goal in Context: The above diagram represents the high-level overview of VisionGuard AI as a single system boundary — "Detect, Enhance and Search" — interacted with by the Admin and Operator actors.

3.1.2 Modules of VisionGuard AI (Level 1)

Level: 1 Use Case ID: 1 Name: VisionGuard AI: Detect, Enhance and Search

Primary Actor: Admin, Operator **Secondary Actor:** None

Level: 1
USE CASE ID: 1
Name: VisionGuard AI: Detect, Enhance and Search
Primary Actor: Admin, Operator
Secondary Actor: None

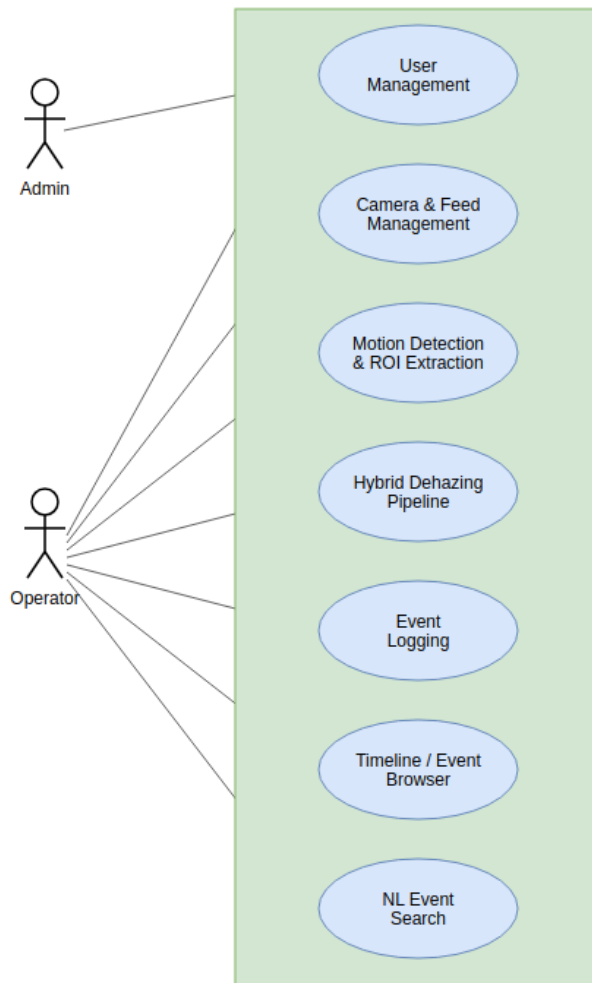


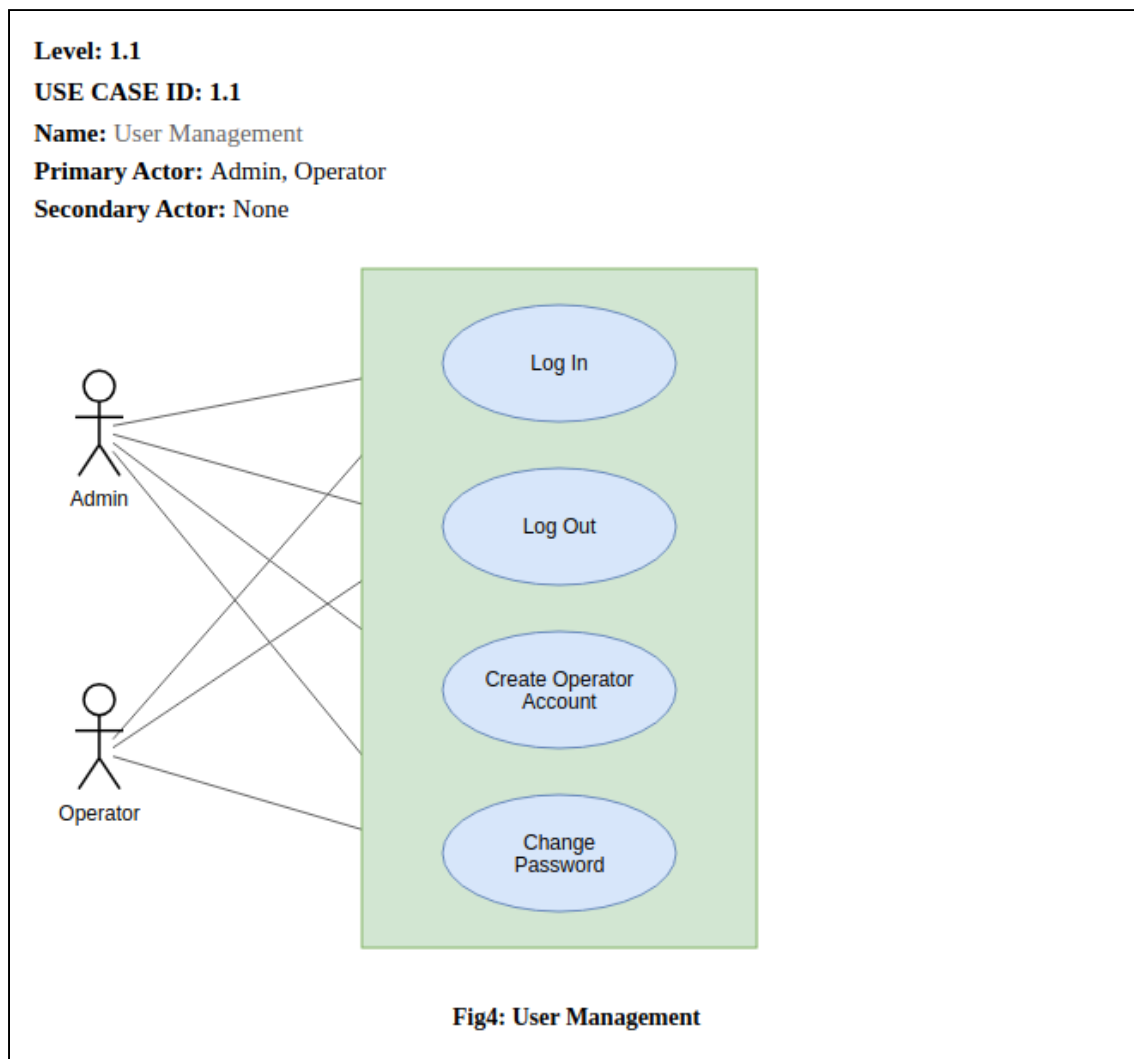
Fig2: VisionGuard AI (Use Case Level-1)

Modules shown: User Management, Camera & Feed Management, Motion Detection & ROI Extraction, Hybrid Dehazing Pipeline, Event Logging, Timeline / Event Browser, NL Event Search.

Goal in Context: The above diagram represents the seven main functional modules that make up VisionGuard AI, with the Operator driving most day-to-day modules and the Admin additionally responsible for account management within User Management.

3.1.3 User Management

Level: 1.1 **Use Case ID:** 1.1 **Name:** User Management **Primary Actor:** Admin, Operator **Secondary Actor:** None



Use cases shown: Log In, Log Out, Create Operator Account, Change Password.

Goal in Context: The above diagram represents the breakdown of the "User Management" component from Use Case Level 1. Both Admin and Operator can log in, log out, and change their own password; only the Admin can create new operator accounts, since VisionGuard AI has no open self-registration.

3.1.4 Camera & Feed Management

Level: 1.2 **Use Case ID:** 1.2 **Name:** Camera & Feed Management **Primary Actor:** Operator **Secondary Actor:** None

Level: 1.2

USE CASE ID: 1.2

Name: Camera & Feed Management

Primary Actor: Operator

Secondary Actor: None

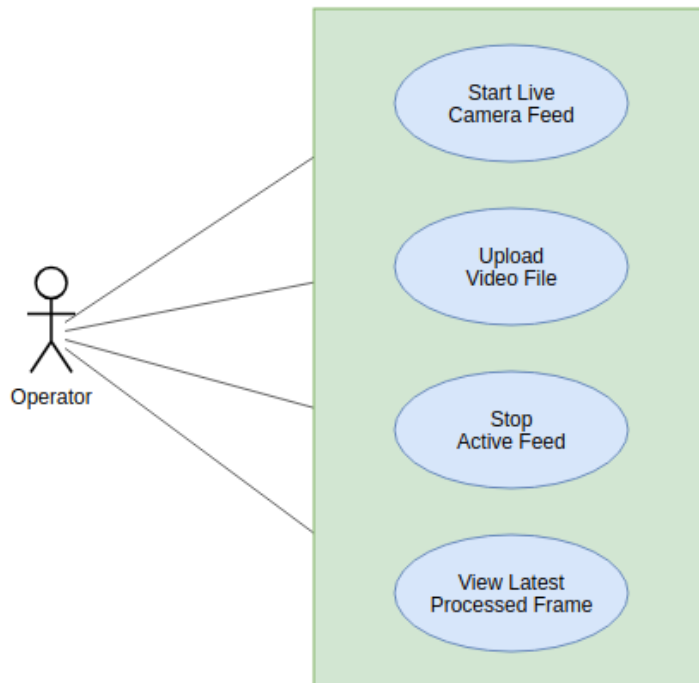


Fig5: Camera & Feed Management

Use cases shown: Start Live Camera Feed, Upload Video File, Stop Active Feed, View Latest Processed Frame.

Goal in Context: The above diagram represents the breakdown of the "Camera & Feed Management" component, covering how an Operator starts a webcam/IP camera feed or uploads a recorded video, stops an active feed, and checks the latest processed frame.

3.1.5 Motion Detection & ROI Extraction

Level: 1.3 Use Case ID: 1.3 Name: Motion Detection & ROI Extraction **Primary Actor:** System (Capture Worker) **Secondary Actor:** None

Level: 1.3

USE CASE ID: 1.3

Name: Motion Detection & ROI Extraction

Primary Actor: System (Capture Worker)

Secondary Actor: None

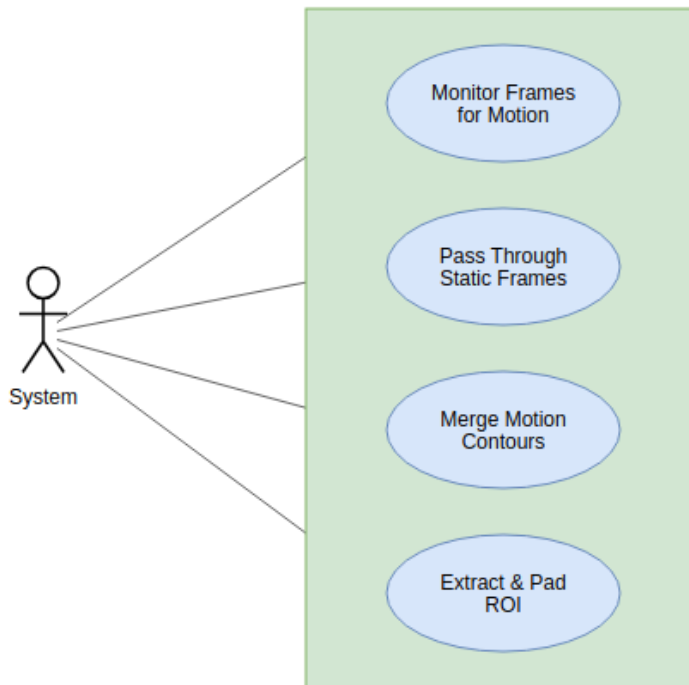


Fig6: Motion Detection & ROI Extraction

Use cases shown: Monitor Frames for Motion, Pass Through Static Frames, Merge Motion Contours, Extract & Pad ROI.

Goal in Context: The above diagram represents the breakdown of the "Motion Detection & ROI Extraction" component. Unlike the previous modules, this one is driven entirely by the system's own capture worker rather than a human actor — it continuously monitors frames, passes static frames through untouched, and merges detected motion contours into a padded ROI when motion is found.

3.1.6 Hybrid Dehazing Pipeline

Level: 1.4 **Use Case ID:** 1.4 **Name:** Hybrid Dehazing Pipeline **Primary Actor:** System (Capture Worker) **Secondary Actor:** None

Level: 1.4

USE CASE ID: 1.4

Name: Hybrid Dehazing Pipeline

Primary Actor: System (Capture Worker)

Secondary Actor: None

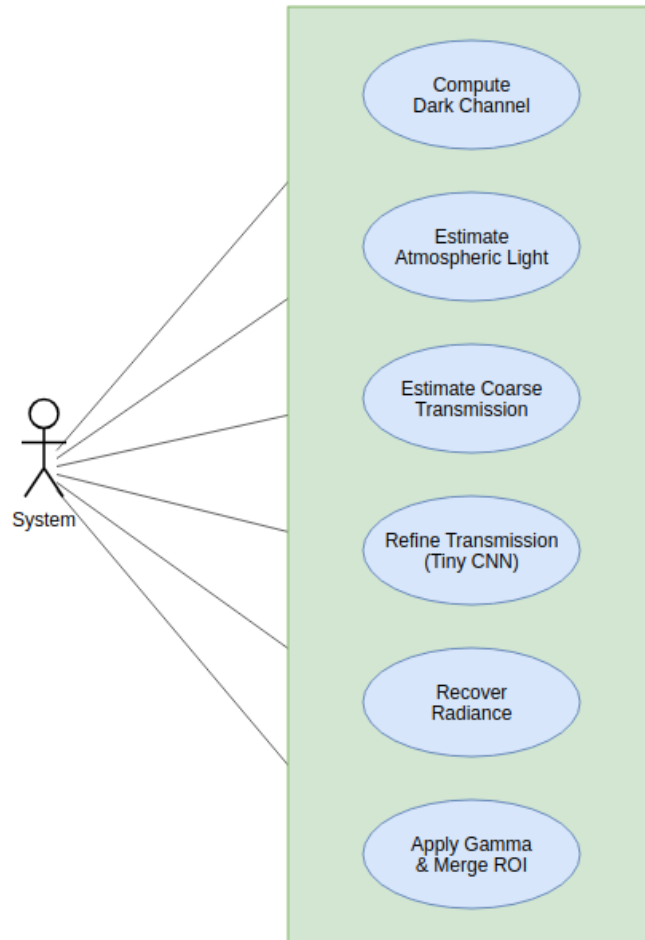


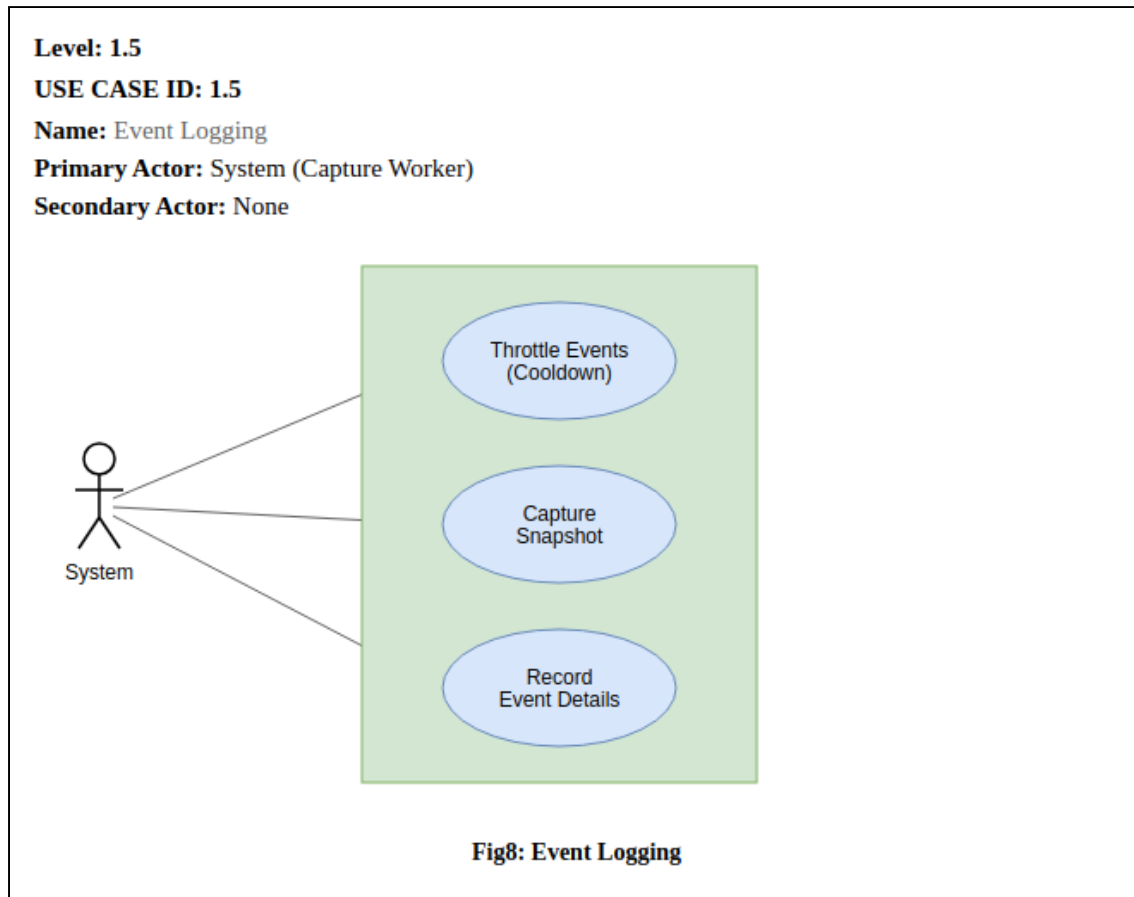
Fig7: Hybrid Dehazing Pipeline

Use cases shown: Compute Dark Channel, Estimate Atmospheric Light, Estimate Coarse Transmission, Refine Transmission (Tiny CNN), Recover Radiance, Apply Gamma & Merge ROI.

Goal in Context: The above diagram represents the breakdown of the core "Hybrid Dehazing Pipeline" component — the physics-and-CNN chain that turns a motion ROI into an enhanced region, matching the Dark Channel Prior + Tiny CNN architecture described in this report's overview.

3.1.7 Event Logging

Level: 1.5 **Use Case ID:** 1.5 **Name:** Event Logging **Primary Actor:** System (Capture Worker) **Secondary Actor:** None



Use cases shown: Throttle Events (Cooldown), Capture Snapshot, Record Event Details.

Goal in Context: The above diagram represents the breakdown of the "Event Logging" component, which throttles repeated logging via a cooldown, captures a snapshot of the enhanced ROI, and records the event's details for later retrieval.

3.1.8 Timeline / Event Browser

Level: 1.6 **Use Case ID:** 1.6 **Name:** Timeline / Event Browser **Primary Actor:** Operator **Secondary Actor:** None

Level: 1.6

USE CASE ID: 1.6

Name: Timeline / Event Browser

Primary Actor: Operator

Secondary Actor: None

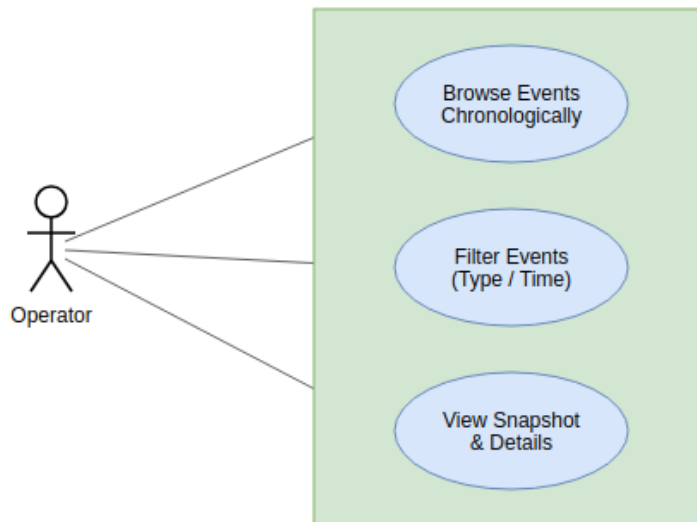


Fig9: Timeline / Event Browser

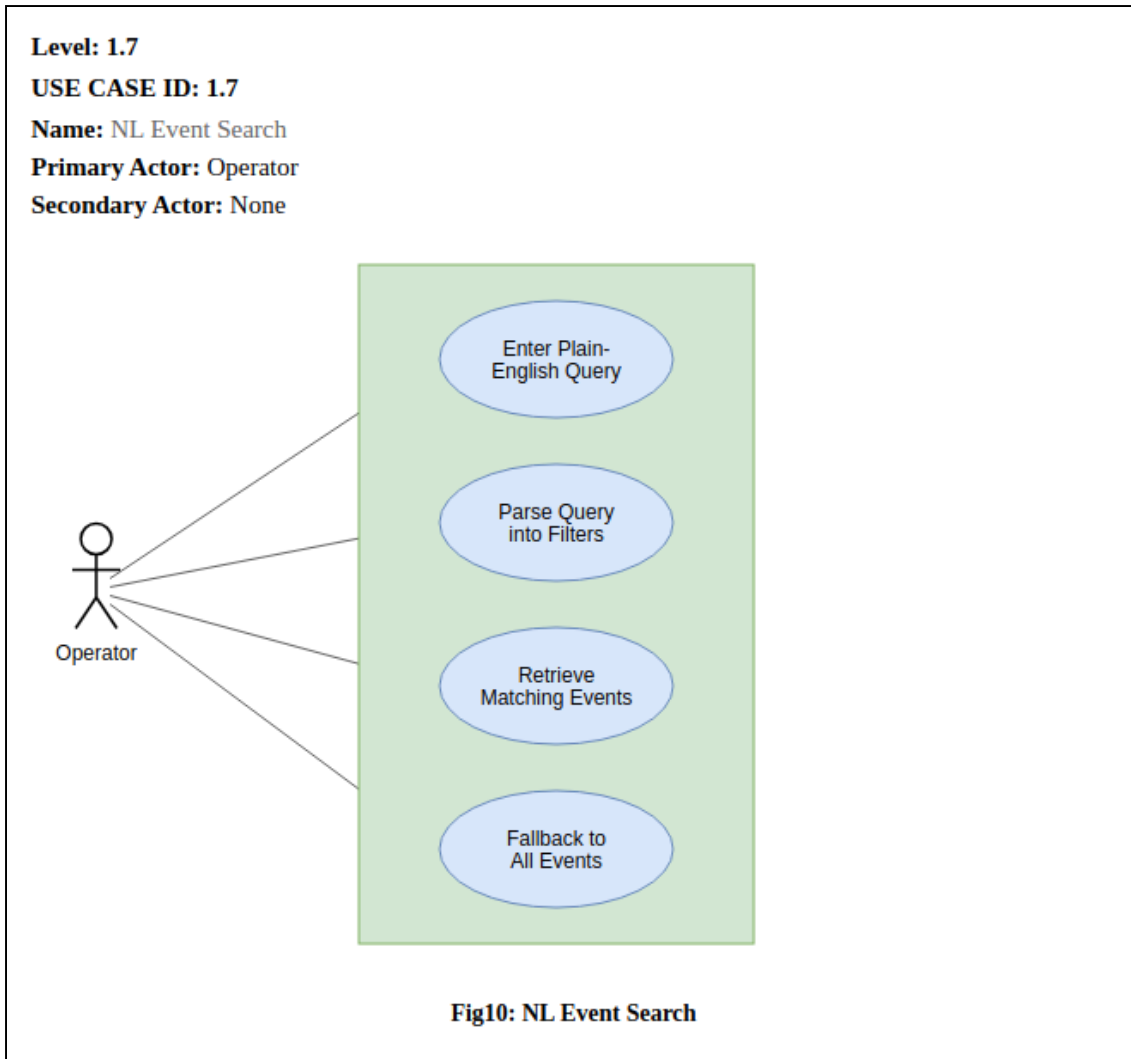
Use cases shown: Browse Events Chronologically, Filter Events (Type / Time), View Snapshot & Details.

Goal in Context: The above diagram represents the breakdown of the "Timeline / Event Browser" component, letting an Operator scan logged events chronologically, filter by type or time, and inspect a specific event's snapshot and details.

3.1.9 NL Event Search

Level: 1.7 Use Case ID: 1.7 Name: NL Event Search **Primary Actor:** Operator

Secondary Actor: None



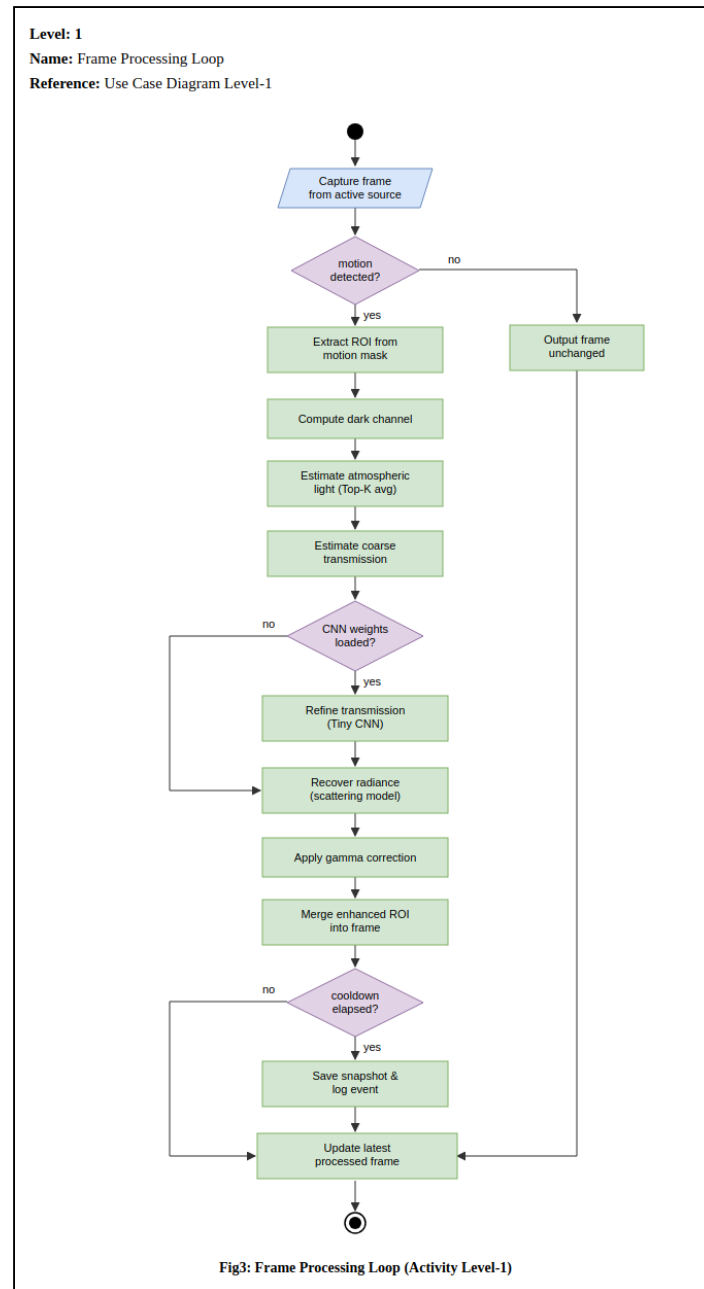
Use cases shown: Enter Plain-English Query, Parse Query into Filters, Retrieve Matching Events, Fallback to All Events.

Goal in Context: The above diagram represents the breakdown of the "NL Event Search" component, where an Operator's plain-English query is parsed into structured filters and used to retrieve matching events, falling back to all events if no filters can be extracted.

3.2 Activity Diagram

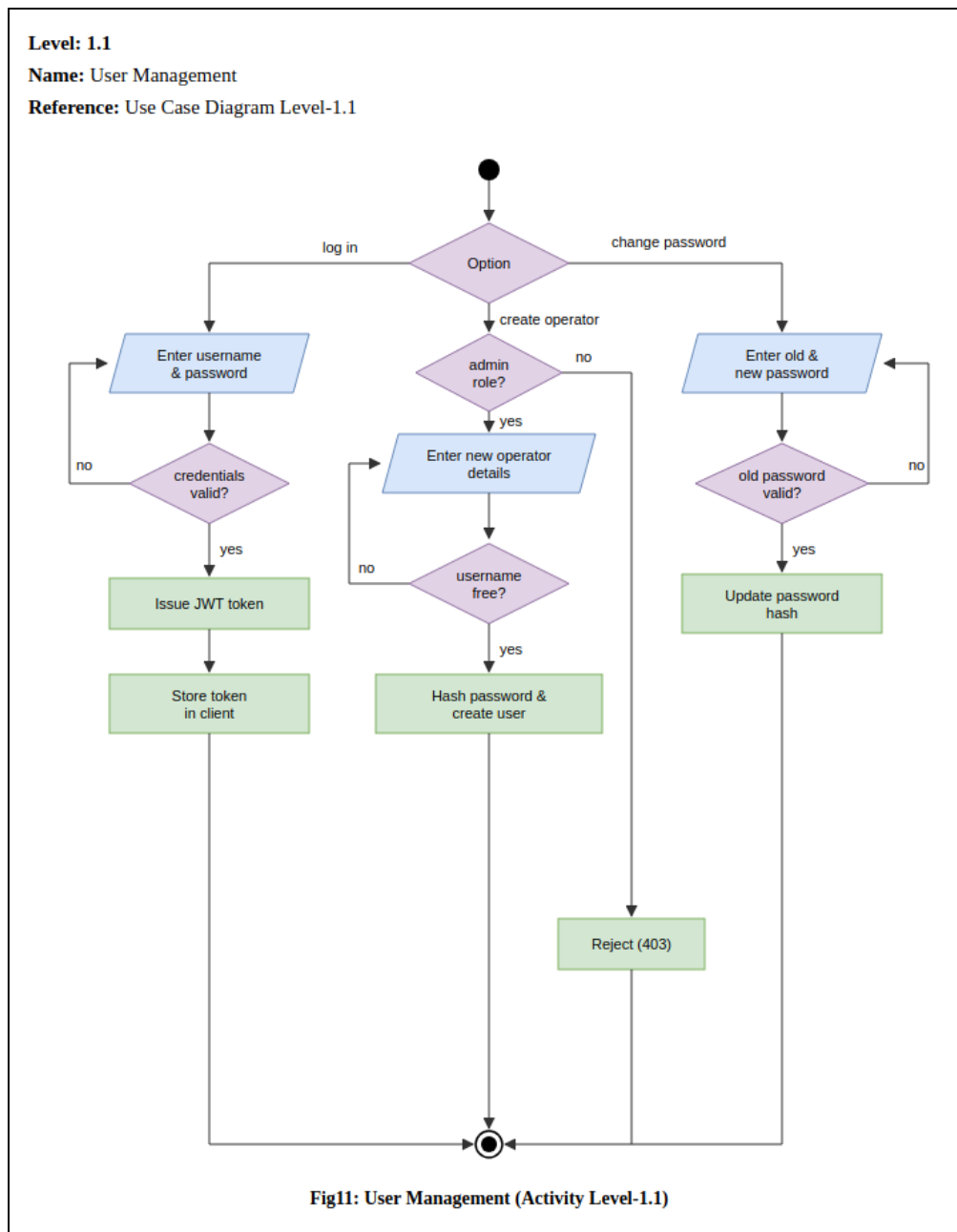
An activity diagram models the flow of control within a use case as a sequence of actions and decision points. The activity diagrams below trace VisionGuard AI's per-frame processing loop first as a single end-to-end flow, then broken down per module to mirror the use case breakdown above.

3.2.1 Frame Processing Loop (Overview)



This is the core real-time loop run by the capture worker for every frame: a frame is captured from the active source, and if no motion is detected the frame is output unchanged. If motion is detected, the ROI is extracted and passed through the dehazing chain — dark channel, Top-K atmospheric light, coarse transmission, CNN refinement (skipped gracefully if weights are not loaded), radiance recovery, and gamma correction — before the enhanced ROI is merged back into the frame. If the event cooldown has elapsed, a snapshot is saved and the event is logged; either way, the latest processed frame is updated for the frontend to read.

3.2.2 User Management



Branches on Log In (validate credentials, issue and store a JWT), Create Operator (rejected with a 403 unless the requester has the admin role, otherwise validates username uniqueness before hashing the password and creating the user), and Change Password (validates the old password before updating the hash).

3.2.3 Camera & Feed Management

Level: 1.2

Name: Camera & Feed Management

Reference: Use Case Diagram Level-1.2

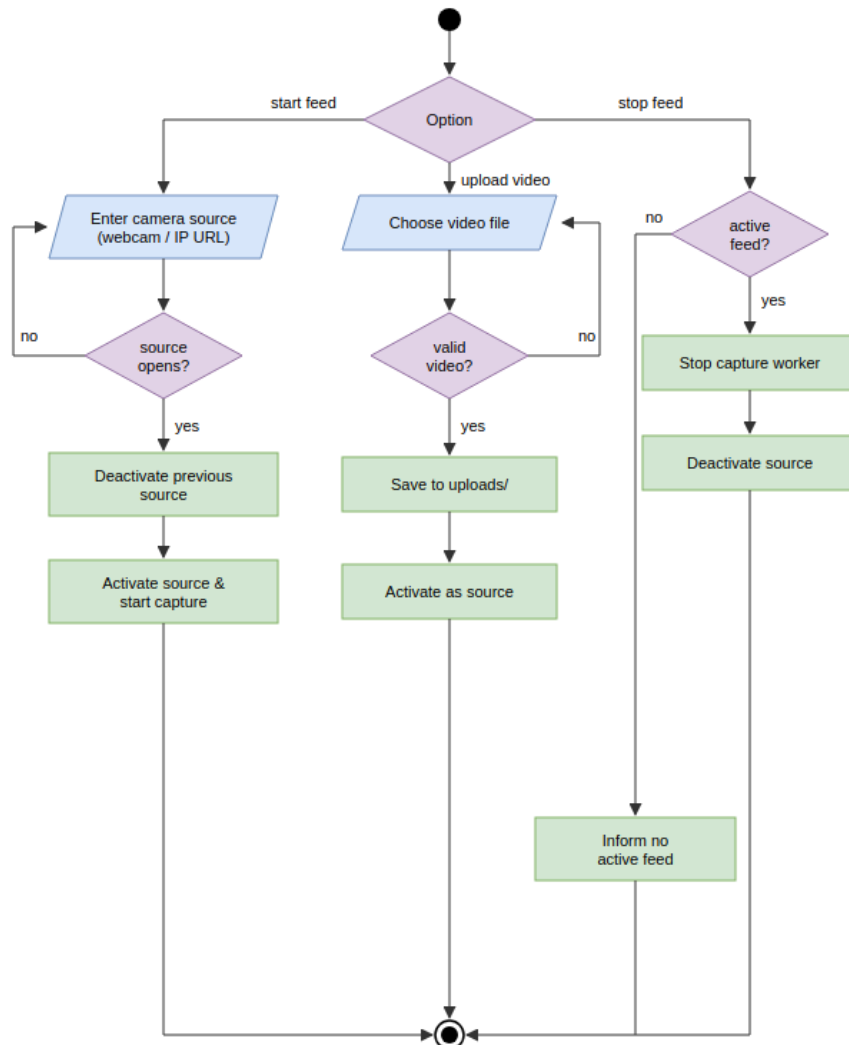


Fig12: Camera & Feed Management (Activity Level-1.2)

Branches on Start Feed (opens the given webcam/IP source, deactivating any previous source before activating the new one), Upload Video (validates the file before saving it to the uploads directory and activating it as the source), and Stop Feed (stops the capture worker and deactivates the source if one is active, otherwise informs the operator there is no active feed).

3.2.4 Motion Detection & ROI Extraction

Level: 1.3

Name: Motion Detection & ROI Extraction

Reference: Use Case Diagram Level-1.3

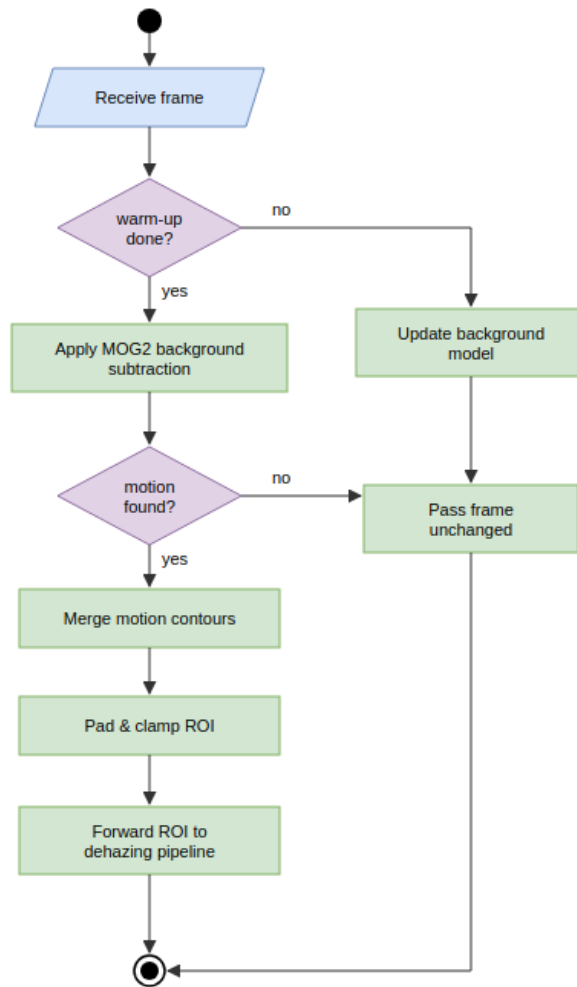


Fig13: Motion Detection & ROI Extraction (Activity Level-1.3)

Each received frame updates the MOG2 background model during a warm-up period; once warm-up is complete, background subtraction runs and, if motion is found, contours are merged and the resulting box is padded and clamped before being forwarded to the dehazing pipeline. If no motion is found, the frame passes through unchanged.

3.2.5 Hybrid Dehazing Pipeline

Level: 1.4

Name: Hybrid Dehazing Pipeline

Reference: Use Case Diagram Level-1.4

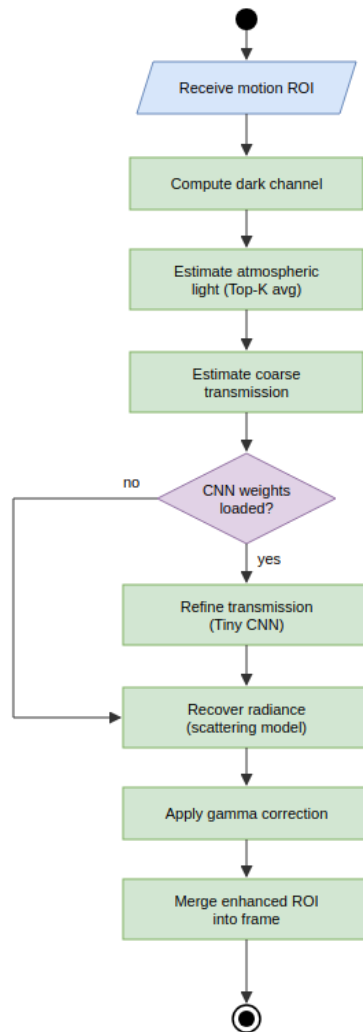


Fig14: Hybrid Dehazing Pipeline (Activity Level-1.4)

Given a motion ROI, the dark channel is computed, atmospheric light is estimated as a Top-K average, and a coarse transmission map is estimated from the Dark Channel Prior. If Tiny CNN weights are loaded, the transmission map is refined by the CNN; otherwise the coarse map is used directly. Radiance is recovered via the atmospheric scattering model, gamma correction is applied, and the enhanced ROI is merged back into the frame.

3.2.6 Event Logging

Level: 1.5

Name: Event Logging

Reference: Use Case Diagram Level-1.5

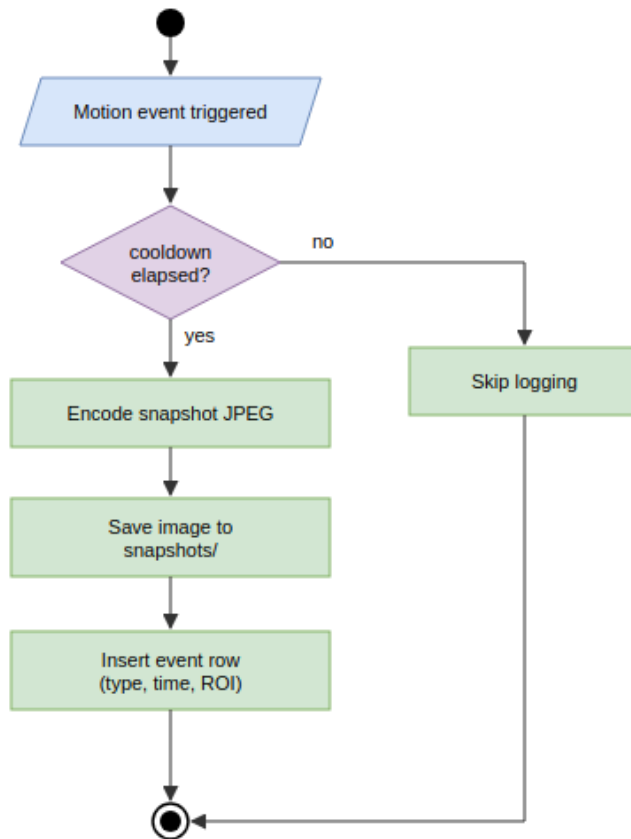


Fig15: Event Logging (Activity Level-1.5)

Triggered whenever a motion event occurs: if the cooldown has not yet elapsed, logging is skipped; otherwise the snapshot is JPEG-encoded, saved to the snapshots directory, and an event row (type, timestamp, ROI) is inserted into the database.

3.2.7 Timeline / Event Browser

Level: 1.6

Name: Timeline / Event Browser

Reference: Use Case Diagram Level-1.6

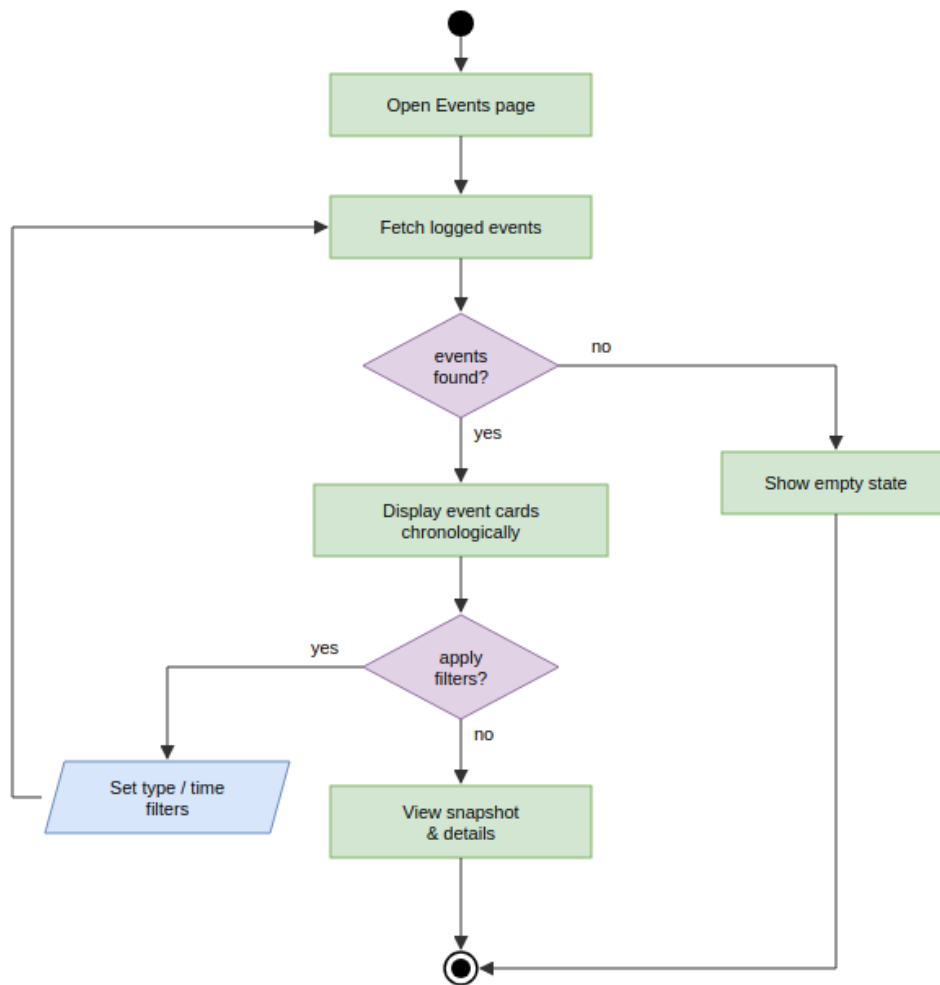


Fig16: Timeline / Event Browser (Activity Level-1.6)

Opening the Events page fetches logged events; if none are found, an empty state is shown, otherwise events are displayed chronologically as cards. The operator may set type/time filters, which re-fetches the event list, or view a specific event's snapshot and details.

3.2.8 NL Event Search

Level: 1.7

Name: NL Event Search

Reference: Use Case Diagram Level-1.7

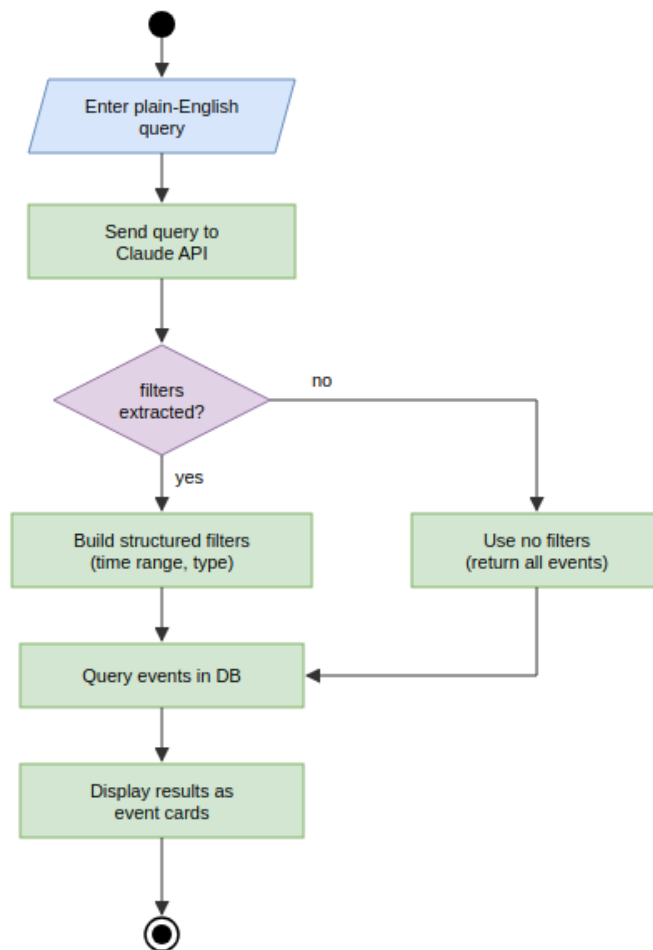


Fig17: NL Event Search (Activity Level-1.7)

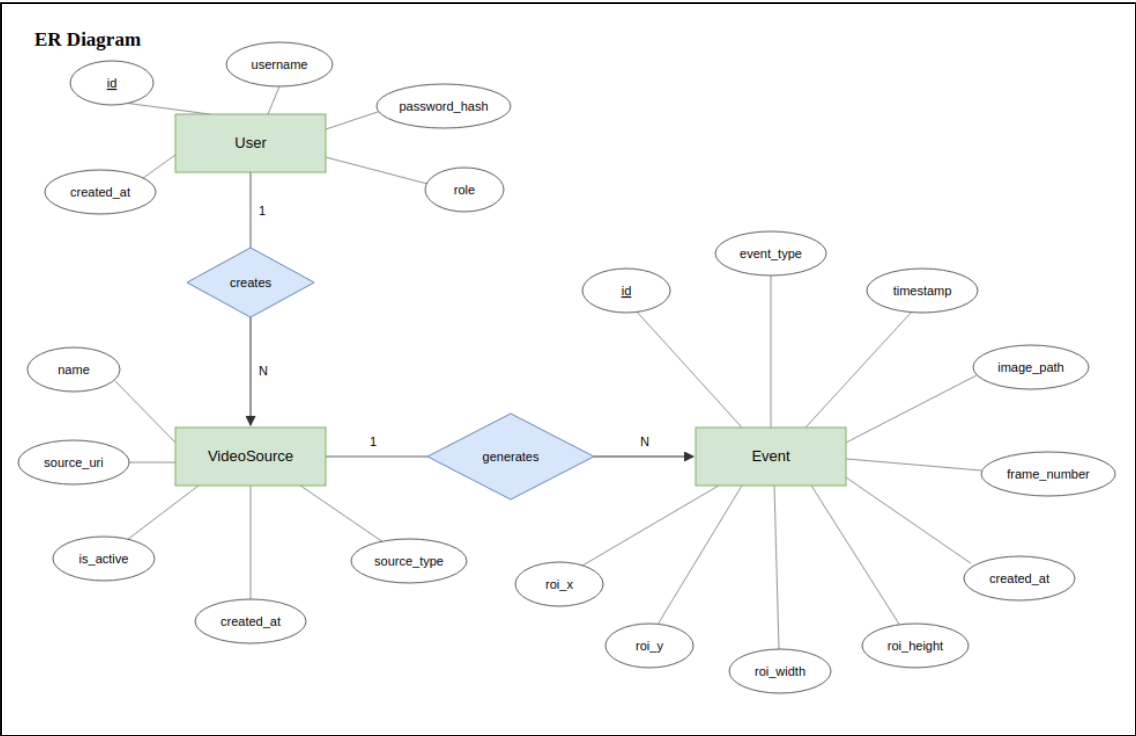
A plain-English query is sent to the Claude API. If structured filters (time range, event type) can be extracted, they are used to query events in the database; if not, the system falls back to querying with no filters rather than failing. Matching results are displayed as event cards.

4. Data & Information Modeling

Option A — Traditional Software Project

ER Diagram

Data-based modeling visually represents a database's structure — its entities, attributes, and the relationships between them — and is built by analyzing the data a system needs to persist. VisionGuard AI persists three entities: the accounts that operate the system, the video sources they configure, and the events those sources generate.



A **User** *creates* many **VideoSource** records (1:N); a **VideoSource** *generates* many **Event** records (1:N). The database schema, including primary and foreign keys, follows:

Table 1: User table for database schema

Attribute Name	Data Type	Key/Constraints
id	INTEGER	Primary Key, autoincrement
username	VARCHAR	Unique, not null
password_hash	VARCHAR	Not null (bcrypt hash, never plaintext)
role	VARCHAR	Not null ("admin" or "operator")
created_at	DATETIME	Not null, server default now

Table 2: VideoSource table for database schema

Attribute Name	Data Type	Key/Constraints
id	INTEGER	Primary Key, autoincrement
name	VARCHAR(255)	Not null
source_type	VARCHAR(20)	Not null (webcam / IP camera / uploaded video)
source_uri	TEXT	Not null
is_active	BOOLEAN	Not null, default false
created_at	DATETIME	Not null, server default now
created_by	INTEGER	Foreign Key → User.id, nullable

Table 3: Event table for database schema

Attribute Name	Data Type	Key/Constraints
id	INTEGER	Primary Key, autoincrement
source_id	INTEGER	Foreign Key → VideoSource.id, not null
event_type	VARCHAR(30)	Not null
timestamp	DATETIME	Not null
image_path	TEXT	Not null
roi_x	INTEGER	Not null
roi_y	INTEGER	Not null
roi_width	INTEGER	Not null
roi_height	INTEGER	Not null
frame_number	INTEGER	Nullable
created_at	DATETIME	Not null, server default now

`events` is indexed on `(source_id, timestamp)` and `(event_type, timestamp)` to keep timeline browsing and natural-language search filters fast as the event log grows.

Option B — AI-Based Project

Dataset Description

The Tiny CNN transmission-refinement model is trained on two complementary data sources rather than a single fixed dataset, since VisionGuard AI's neural component only needs to learn how to correct the Dark Channel Prior's errors, not to generate images from scratch.

- **Dataset source:**
- A **synthetic haze generator** that procedurally creates training samples: a random textured clean scene and a smooth, random ground-truth transmission field are combined through the atmospheric scattering model ($I = J \cdot t + A \cdot (1-t)$) to synthesize a hazy image. The network's input channels (grayscale ROI, dark channel, coarse transmission) are computed using the project's actual DCP implementation, so the CNN learns to correct that specific estimator's real errors rather than an idealized one.
- The **REVIDE** real-world paired haze dataset (CVPR 2021), which records the same indoor scenes with and without machine-generated haze, giving pixel-aligned hazy/clear frame pairs for validating the model on real haze rather than only synthetic haze.
- **Number of samples:** 3,000 synthetic samples per training epoch by default (effectively unlimited and reproducible, generated deterministically per RNG seed/index), combined with 42 REVIDE training scene folders (1,697 hazy training images) and 6 REVIDE test scene folders (284 hazy test images) used for real-data validation.
- **Features:** each sample's input is a 3-channel stack — grayscale hazy ROI, dark channel, and coarse DCP transmission map — matching the Tiny CNN's documented input in this report's pipeline description.
- **Labels:** there is no classification label; the training target is a refined transmission map. For synthetic samples the ground-truth transmission field is known exactly; for REVIDE's real pairs (which have no true transmission map) the coarse DCP transmission is used as a physically-grounded anchor. The model is optimized primarily through a self-supervised **image reconstruction loss** — the L1 distance between the image recovered using the predicted transmission

and the true clean image — with a small auxiliary L1 term against the known transmission on synthetic data.

- **Data preprocessing:** images are cropped to 96×96 training patches, the DCP is computed with a 15-pixel erosion window, and inputs/targets are normalized to `[0, 1]` float32 before being batched (batch size 16) for training with Adam (learning rate 1e-3, halved every 8 epochs) over 15 epochs by default, with the best-validation-loss weights saved to `backend/weights/tiny_cnn.pth`.
-

5. Preliminary Test Plan

5.1 Testing Objectives

- Verify that all functional requirements listed in Section 2.1 behave as specified.
- Ensure the motion-gated, ROI-only dehazing pipeline remains stable under continuous frame processing.
- Validate that error and edge-case handling (missing CNN weights, invalid uploads, unparseable search queries) degrades gracefully instead of crashing.
- Evaluate whether the pipeline meets its real-time performance target on CPU-only hardware.

5.2 Features to be Tested

Table 4: High-level testing plan for VisionGuard AI

#	Title	Scenario	Steps	Expected Outcome
1	Operator Login	Verify secure login flow.	1. Enter valid username/password. 2. Submit <code>POST /auth/login</code> .	A signed JWT access token is issued and the frontend stores it for subsequent requests.
2	Admin-Only Account Creation	Restrict operator account creation to admins.	1. Log in as an operator. 2. Attempt <code>POST /auth/users</code> .	Request is rejected (403); no row is created in the User table.
3	Live Feed Start	Start a webcam/IP camera feed.	1. Call <code>POST /start-camera</code> with a source URI. 2. Check <code>/frame</code> .	The capture worker activates the source and <code>/frame</code> returns a processed frame.
4	Video Upload Validation	Upload a video file for processing.	1. <code>POST /upload-video</code> with an invalid (non-video) file.	The operator receives a clear validation error rather than a silently broken stream.
5	Motion-Skip Behavior	Confirm dehazing is skipped when idle.	1. Feed static frames with no motion.	The dehazing pipeline is never invoked; frames pass through unchanged, keeping CPU usage low.
6	ROI Extraction Correctness	Confirm only the motion region is processed.	1. Feed a frame with a single moving object.	The extracted ROI bounding box tightly (with padding) covers the motion contour, not the full frame.
7	Dehazing Reconstruction Quality	Verify the hybrid pipeline improves visibility.	1. Run a hazy ROI through the full pipeline (DCP → CNN → radiance recovery → gamma).	The recovered ROI has measurably higher contrast/lower haze than the coarse-DCP-only output.
8	CNN Weights Missing Fallback	Ensure pipeline degrades gracefully without trained weights.	1. Start the pipeline with <code>MODEL_PATH</code> pointing to a missing file.	The pipeline falls back to the unrefined DCP transmission instead of crashing.
9	Event Logging & Cooldown	Verify events are throttled, not logged every frame.	1. Trigger continuous motion for several seconds.	Exactly one event is logged per cooldown window, each with correct timestamp, image

#	Title	Scenario	Steps	Expected Outcome
				path, and ROI coordinates.
10	Timeline Browsing	Verify chronological event retrieval.	1. Log several events. 2. Call <code>GET /events</code> .	Events are returned in chronological order with snapshot and metadata intact.
11	NL Event Search — Time Parsing	Verify natural language queries resolve to filters.	1. Call <code>GET /events/search?q=any motion last night?</code> .	Claude API returns a parsed time range; only events within that range are returned.
12	NL Event Search — Fallback	Verify unparseable queries do not fail.	1. Submit a vague query with no extractable time/type.	The system returns all events rather than raising an error.
13	System Status Check	Confirm health endpoint reflects real pipeline state.	1. Call <code>GET /system-status</code> while a feed is active.	Response reports the camera feed and dehazing pipeline as running.
14	Feed Stop Without Active Source	Guard against invalid stop requests.	1. Call <code>POST /stop-camera</code> with no active feed.	The operator is informed no feed is active rather than the server raising an unhandled error.
15	Auth Enforcement on Protected Routes	Confirm router-level auth guards all non-login endpoints.	1. Call <code>GET /events</code> without an Authorization header.	Request is rejected (401) before reaching any controller or service logic.